System Hardening & Windows Endpoint Security: Networking

4/18/26 11:05 AM

Summary:

- **Hardening**: Disable/remove what isn't needed (services, roles, accounts), enforce least privilege, use AppLocker/firewall.
- **Networking investigation**: Always check ARP cache, routing table, active connections, DNS cache, and hosts file.
- **Detection workflow**:
  1. Baseline with **nmap** or **ipconfig**.
  2. Disable unnecessary services/roles.
  3. Investigate anomalies (netstat -ano, Test-NetConnection, hosts file).
  4. Harden firewall and AppLocker rules.

Common Tools/Commands:

- **nmap**: Baseline open ports.
- **ipconfig /all**, **route print**, **netstat -ano**.
- **Test-NetConnection**: Port and connectivity checks.
- **nslookup**: DNS investigation.
- **ufw** (Linux firewall), **AppLocker** (Windows application control).

1. Hardening Linux Systems (Ubuntu Web Server)

Objective: Remove unnecessary services, disable root SSH, and enable firewall while keeping required web services.

Step-by-Step Hardening:

1. **Baseline scan**:
   Bash
   ```
   nmap 172.31.24.10
   ```
   - Expected open ports: 22 (SSH), 80/443 (HTTP/HTTPS), 389 (LDAP — unnecessary).
2. **Disable direct root SSH**:
   Bash
   ```
   sudo vi /etc/ssh/sshd_config
   ```
   - Change PermitRootLogin yes to PermitRootLogin no.
   - Save & exit: :wq
   - Restart SSH:
     Bash
     ```
     sudo systemctl restart sshd
     ```
3. **Disable unnecessary service (LDAP example)**:
   Bash
   ```
   sudo systemctl disable --now slapd.service
   sudo apt purge slapd          # Remove package and dependencies
   ```
4. **Enable and configure firewall**:
   Bash
   ```
   sudo ufw allow "Apache Full"   # Allow HTTP/HTTPS
   sudo ufw allow OpenSSH         # Allow SSH
   sudo ufw enable
   sudo ufw status
   ```

Verification:
Bash
```
nmap 172.31.24.10
```

- Port 389 should no longer be open.

Key takeaway: Always baseline with **nmap**, disable what isn't needed, and explicitly allow only required services in the firewall.

2. Hardening Windows Systems (DNS Server)

Objective: Remove unnecessary roles, harden default admin account, disable unneeded services, and restrict applications with AppLocker.

Step-by-Step Hardening:

1. **Baseline scan**:
   Bash
   ```
   nmap -Pn 172.31.24.20
   ```
   - Expected: 53 (DNS), 80 (HTTP — unnecessary), 135/3389 (RDP — lab only).
2. **Remove unnecessary role (IIS)**:
   - Server Manager → Manage → Remove Roles and Features.
   - Deselect **Web Server (IIS)** → Restart server.
3. **Harden default Administrator account**:
   - Computer Management → Local Users and Groups → Users.
   - Rename Administrator to **psadmin**.
   - Set strong password.
   - Disable unneeded accounts (e.g., WDAGUtilityAccount).
4. **Disable unnecessary services**:
   - Services.msc
   - Set **ActiveX Installer**, **Downloaded Maps Manager**, **Print Spooler** to **Disabled**.
5. **Enable Application Identity service** (required for AppLocker):
   - Set to Automatic and start it.
6. **Restrict applications with AppLocker**:
   - Local Security Policy → Application Control Policies → AppLocker → Executable Rules.
   - Automatically Generate Rules for C:\Program Files.
   - Create Deny rule for specific apps (e.g., Wireshark) to test blocking.

Verification:
Bash
```
nmap -Pn 172.31.24.20
```

- Port 80 should no longer be open.

Key takeaway: Remove unneeded roles/services, rename/disable default admin accounts, and use **AppLocker** to control what can run.

3. Windows Endpoint Security: Networking Fundamentals

Goal: Use native Windows tools to investigate networking without third-party software ("living off the land").

Key Commands & What They Show:

- **arp -a**: View ARP cache (IP-to-MAC mappings).
  - Add entry (for testing): arp -s 192.2.3.4 00-aa-00-62-c6-09
- **ipconfig**:
  - ipconfig /all → Full config (IP, gateway, DNS, MAC).
  - ipconfig /flushdns → Clear DNS cache.
  - ipconfig /registerdns → Register with DNS server.
- **route print**: View routing table (default gateway, persistent routes).
  - Malware can add routes for redirection.
- **ping**:
  - Test connectivity: ping 172.31.24.32
  - Options: -n 2 (send 2 packets), -t (continuous).
- **netstat -ano**:
  - Show active connections, listening ports, owning process ID.
  - Useful for spotting C2 or unexpected listeners.
- **Test-NetConnection** (PowerShell):
  - Test-NetConnection -computername badserver
  - Test-NetConnection -computername badserver -port 3389 → Test specific port.
  - -informationlevel Detailed → More info.

Hosts File (C:\Windows\System32\drivers\etc\hosts):

- Local override for DNS (maps domain to IP).
- Attackers modify it for redirection/phishing.
- Example: Add 172.31.24.32 badserver → resolves without real DNS.

nslookup:

- nslookup google.com → Resolve name.
- Inside nslookup: set type=mx → Show mail servers.
- set type=ANY → Show all records.

Security relevance:

- Check for unexpected routes, poisoned DNS cache, suspicious listening ports, or modified hosts file.
- High ephemeral ports with low packet count = normal.
- One port with massive traffic = possible C2.

Key takeaway: Use native tools (ipconfig, netstat, Test-NetConnection, nslookup) to baseline and detect anomalies without installing software.

IPv4 and ARP

4/15/26 2:31 PM

Summary:

- **Layer 2 (MAC/ARP)**: Local, easily spoofed → monitor switch logs and **ARP** for **MITM** or reconnaissance.
- **Layer 3 (IPv4)**: Routable but spoofable → focus on **TTL anomalies**, source validation, and abnormal traffic.
- **Detection workflow**:
  1. Capture traffic (**Wireshark** / **tcpdump** / **Suricata**).
  2. Filter anomalies (**ARP** floods, low/varying **TTL**, unusual protocols).
  3. Correlate with host artifacts (processes, logs).
  4. Block suspicious sources and investigate root cause.

  Common Tools & Commands:
- **Wireshark**: Packet inspection (arp, ip.ttl, eth.src filters).
- **tcpdump**: Command-line capture (tcpdump -e -nn -r file.pcap).
- **arp -a**: View **ARP** cache.
- **Suricata**: Rule-based detection of **ARP** storms or **IPv4** anomalies.
  These fundamentals help you quickly spot **Layer 2/3** attacks in real incidents (e.g., **ARP** poisoning during data exfil or **TTL** changes indicating **MITM**).

1. MAC Addressing (Layer 2)

What is a MAC address? 48-bit hardware identifier for network interfaces (e.g., **00:1A:2B:3C:4D:5E**). First 3 bytes = manufacturer (**OUI**), last 3 bytes = device-specific.
Security relevance: MAC is used only for local network communication. It is **not routable** and **easily spoofed**. Attackers change MAC to evade tracking, bypass filters, or impersonate devices.
Spoofing MAC Addresses (common attack):

- Attacker changes their MAC to look trusted or hide identity.
- Linux example:
  Bash
  ```
  sudo ip link set eth0 down
  sudo ip link set eth0 address 00:11:22:33:44:55
  sudo ip link set eth0 up
  ```

Detection in IR:

- Sudden **MAC changes** for the same IP (check switch logs or ARP tables).
- Multiple devices claiming same MAC → spoofing or conflict.
- **Wireshark filter**: eth.src == xx:xx:xx:xx:xx:xx.

Key takeaway: Never trust **MAC** as a security control. Treat it as untrusted in investigations.

2. Address Resolution Protocol (ARP)

What ARP does: Maps **IP address** (Layer 3) to **MAC address** (Layer 2) on the local network.
Normal ARP flow:

1. Host checks **ARP cache** (arp -a).
2. If missing, broadcasts **ARP Request** (destination MAC = all **FF:FF:FF:FF:FF:FF**).
3. Target replies with unicast **ARP Reply** containing its MAC.
4. Sender updates cache and sends data.

ARP Attacks:

- **ARP Poisoning (MITM)**: Attacker sends fake ARP Replies claiming wrong MAC for an IP. Traffic is redirected through attacker.
- **ARP Storm/Flood**: Attacker floods network with ARP requests → DoS (overloads switches/hosts). One MAC pretending to own many IPs.

Detection:

- **ARP storm**: Sudden spike in ARP requests (**Wireshark filter** arp or Suricata rules).
- **Poisoning**: Same IP mapped to multiple MACs or inconsistent replies.

Demo: ARP Storm Detection:

- Use **tcpdump** or **Wireshark** on PCAP.
- Command example:
  Bash
  ```
  tcpdump -e -nn -r arp-storm.pcap | cut -d' ' -f2,10 | cut -d'.' -f1-4 | sort | uniq -c | sort -n
  ```
- Look for one **MAC** associated with many IPs (anomalous).

Key takeaway: **ARP** has no authentication. Monitor for floods or poisoning in any suspicious network incident.

3. IPv4 and Malicious Traffic

IPv4 Basics: 32-bit logical address (e.g., **192.168.1.10**). Header includes **Source IP**, **Destination IP**, **TTL**, Protocol, etc. Unlike MAC, **IPv4** is routable across networks.
Time To Live (TTL):

- Starts at OS default (**Windows = 128**, **Linux = 64**, routers often **255**).
- Decrements by 1 per router hop.
- Reaches 0 → packet dropped (prevents loops).
- **Security use**: Baseline normal TTL. Anomalies help detect **spoofing**, **tunneling**, or **MITM** (extra hop).

Detecting Anomalies Using TTL (Demo):

- In **Wireshark**: Add **TTL** column or filter ip.ttl < 50.
- Normal: Consistent high TTL for same source.
- Suspicious: Sudden drop (e.g., from 47 to 46) or unexpectedly low TTL → possible extra device (**MITM**) or spoofing.
- Example: Client packet **TTL=128** (Windows), server reply **TTL=47** → suggests ~17 hops. Sudden change to 46 = investigate as potential **MITM**.

Other Malicious IPv4 Patterns:

- **IP spoofing** (fake source IP).
- **Fragmentation attacks** (overlapping fragments to evade IDS).
- Abnormal packet sizes or unusual protocols from one source.

Detection & IR Response:

- Use **ACLs/firewalls** to validate source IPs.
- Monitor **TTL anomalies** and spoofed traffic.
- Correlate with host logs (e.g., unexpected processes making outbound connections).
- In **Wireshark**: Filter by source IP and watch **TTL** changes.

Security Best Practices:

- Understand your network's normal routing behavior.
- Block unexpected traffic patterns.
- Move toward better segmentation and **IPv6** where possible (stronger security features).

Key takeaway: **IPv4** has weak built-in security. Use **TTL**, header inspection, and traffic patterns to spot anomalies like spoofing, tunneling, or hidden C2 during incident response.

Zeek

4/20/26 6:50 PM

What is Zeek? Zeek is a passive, scriptable network analysis framework. It parses protocols in real time, generates structured logs, and lets you write custom scripts to detect threats.
Zeek vs Arkime (quick):

- Arkime = fast PCAP search and storage.
- Zeek = deep protocol analysis + custom scripting for alerts.

Companies often run both together.

Zeek System Workflow

1. Packets arrive → Event Engine creates events (http_request, dns_request, etc.).
2. Policy scripts react to those events.
3. Notices, logs, or actions are generated.

Key Directories:

- /usr/local/zeek/share/zeek/base/ — core frameworks.
- /usr/local/zeek/share/zeek/policy/ — built-in policies.
- /usr/local/zeek/share/zeek/site/ — your custom scripts (local.zeek).

Basic Commands:
Bash

```
zeek -r capture.pcap local.zeek    # Replay PCAP with your script
zeek -i eth0                       # Live capture
zeekctl deploy                     # Start cluster after editing node.cfg
zeekctl status                     # Check running status
```

Writing Zeek Scripts – Structure & Keys

Basic Script Template (with line-by-line comments):

```
zeek
@load base/protocols/http        # Load the HTTP protocol framework so we can use its events
module MyModule;                 # Optional: organizes your script into a module (good practice)
export {                         # Export section makes things visible to other scripts
   redef enum Notice::Type += { Suspicious_HTTP };   # Add a new notice type so we can raise alerts
}
event http_request(c: connection, method: string, original_URI: string, ...)
   # This event fires every time Zeek sees an HTTP request
   {
   if ("password" in original_URI)   # Check if the word "password" appears in the URI
      {
      NOTICE([$note = Suspicious_HTTP,      # Raise a notice/alert with our custom type
            $msg = "Possible password in URI",  # Human-readable message
            $conn = c]);                # Attach the connection details
      }
   }
```

Key Elements:

- @load — includes frameworks or other scripts.
- event — handler that runs when something happens (e.g., http_request).
- NOTICE — raises alerts (shows in notice.log).
- redef — modifies built-in constants or enums.

Data Types You Need:

- set[addr] — unique IPs.
- table[addr] of count — counters per IP.
- vector of string — lists.

Events & Handling

Most Useful Events:

```
zeek
event new_connection(c: connection)        # Any new connection
event http_request(...)                    # HTTP request
event dns_request(...)                     # DNS query
event ssh_login_attempt(...)               # SSH login try
event connection_state_remove(c: connection) # Connection ended
```

Example: Count failed SSH logins (with comments):

```
zeek
global ssh_fails: table[addr] of count &default=0;   # Table that counts failures per IP address
event ssh_login_attempt(c: connection, success: bool)
   # This event fires every time an SSH login attempt happens
   {
   if (!success)                           # If the login failed
      ssh_fails[c$id$orig_h] += 1;            # Increase the failure count for this source IP
if (ssh_fails[c$id$orig_h] > 5)            # If failures exceed threshold
      NOTICE([$note = SSH_Brute_Force,         # Raise alert
            $msg = "Brute force detected",
            $conn = c]);
   }
```

Notice Framework (Alerts)

How to raise a notice:

```
zeek
NOTICE([$note = My_Alert,              # The type of notice
      $msg = "Something suspicious",    # Human-readable message
      $conn = c]);                  # Attach connection details
```

Notices go to notice.log and can be emailed or sent to SIEM.

Logging Framework

Basic log creation (with comments):

```
zeek
redef enum Log::ID += { LOG };          # Add a new log stream ID
type Info: record {                     # Define what fields will be in the log
   ts: time &log;                       # Timestamp (logged)
   src: addr &log;                      # Source IP (logged)
   msg: string &log;                    # Message (logged)
};
event zeek_init()                       # Runs once when Zeek starts
   {
   Log::create_stream(LOG, [$columns=Info, $path="mylog"]);  # Create the log file "mylog.log"
   }
```

Write to log:

```
zeek
Log::write(LOG, [$ts=network_time(), $src=c$id$orig_h, $msg="Alert"]);  # Write a record to the log
```

Monitoring DNS (Common Example)

```
zeek
event dns_request(c: connection, query: string)
   # Fires every time Zeek sees a DNS query
   {
   if (length(query) > 50 || /\.ru$/ in query)   # High entropy or suspicious TLD
      {
      NOTICE([$note = Suspicious_DNS,
            $msg = "Possible C2 DNS",
            $conn = c]);
      }
   }
```

Best Practices & Optimization

Script Writing Rules:

- Keep scripts small and focused.
- Use @load only for what you need.
- Always test with zeek -r small.pcap local.zeek.
- Add thresholds to avoid noise (e.g., >5 failed logins).
- Comment clearly.

Tuning Tips:

- Use &default=0 on tables.
- Avoid tight loops.
- Use timers for cleanup.
- Recompile only when needed (zeekctl restart).

Example Optimized Script (failed SSH) — with comments:

```
zeek
global ssh_fails: table[addr] of count &default=0;   # Counter per IP, default 0
event ssh_login_attempt(c: connection, success: bool)
   # Fires on every SSH login attempt
   {
   if (!success)                           # If login failed
      {
      ssh_fails[c$id$orig_h] += 1;            # Increment failure count for this source IP
if (ssh_fails[c$id$orig_h] > 5)          # If more than 5 failures
      NOTICE([$note = SSH_Brute_Force,      # Raise alert
            $msg = "Brute force detected",
            $conn = c]);
      }
   }
```

Zeek Base Protocols & Frameworks – Quick Reference

1. Zeek Base Protocols

These are the core protocol parsers. You @load them to get events for common traffic.
Most Important Ones You Need to Know:

Protocol	What it does	Common Use in Scripts
http	Parses HTTP requests/responses	Detect suspicious URIs, POSTs, User-Agents
dns	Parses DNS queries and responses	Detect C2, tunneling, high-entropy domains
ssh	Parses SSH connections and login attempts	Detect brute force
ssl / tls	Parses TLS handshakes and certificates	Detect unusual JA3, self-signed certs
smtp	Parses email traffic	Detect spam or data exfil via email
ftp	Parses FTP commands and file transfers	Detect data exfiltration
rdp	Parses Remote Desktop Protocol	Detect brute force or lateral movement

How to load them:

```
zeek
@load base/protocols/http
@load base/protocols/dns
@load base/protocols/ssh
```

2. Zeek Base Frameworks

These provide the tools and structure for writing useful scripts (noticing, logging, statistics, etc.).
Most Important Frameworks You Need to Know:

Framework	What it does	Common Use in Scripts
notice	Generates alerts/notices	Raise alerts (e.g., brute force, C2)
logging	Creates custom log files	Write your own structured logs
signatures	Signature-based detection	Load .sig files for simple pattern matching
sumstats	Tracks statistics and thresholds	Count events (e.g., failed logins) and alert
cluster	Manages distributed Zeek deployment	Run Zeek across multiple sensors
broker	Communication between Zeek nodes	Share data between workers and managers
input	Reads external files (CSV, JSON, etc.)	Load IOC lists or config files
intelligence	IOC matching (IPs, domains, etc.)	Match traffic against known bad indicators
analyzer	Core protocol analysis engine	Enable/disable analyzers

How to load them:

```
zeek
@load base/frameworks/notice
@load base/frameworks/logging
@load base/frameworks/sumstats
```

Most Common Pattern You Will Use:

```
zeek
@load base/protocols/http        # Get HTTP events
@load base/frameworks/notice     # Get the ability to raise alerts
event http_request(...)          # React to HTTP traffic
{
NOTICE([...]);                # Raise an alert
}
```

TCP and UDP

4/15/26 8:51 PM

Summary:

- **TCP**: Reliable but visible (handshake + flags). Use for detecting scans, C2, and data exfil.
- **UDP**: Fast and lightweight. Use for detecting tunneling, amplification, and fast exfil.
- **Detection workflow**:
  1. Capture traffic (**Wireshark** / **tcpdump**).
  2. Filter for anomalies (**SYN** floods, unusual ports, flag patterns).
  3. Correlate with host artifacts (processes, logs).
  4. Investigate root cause and block.

Common Commands:

- tcpdump -nn -r file.pcap — read PCAP.
- tcpdump -nn -r file.pcap 'tcp[tcpflags] == tcp-syn' — SYN packets only.
- tcpdump -nn -r file.pcap 'src host IP' | cut ... | sort | uniq -c — analyze ports and counts.

These basics let you quickly spot Layer 4 attacks in real incidents (e.g., SYN scans, C2 over common ports, or UDP tunneling).

1. TCP vs UDP Basics

TCP (Transmission Control Protocol):

- **Connection-oriented** — establishes reliable connection before sending data.
- Uses **three-way handshake**: SYN → SYN-ACK → ACK.
- Guarantees delivery with sequence/acknowledgment numbers.
- Higher overhead, slower, but reliable (used for web, email, file transfers).

UDP (User Datagram Protocol):

- **Connectionless** — sends data without handshake or guaranteed delivery.
- Very low overhead, fast (used for DNS, NTP, streaming, VoIP).
- No sequence numbers or acknowledgments — “fire and forget”.

Security relevance:

- **TCP** is commonly abused for scanning, C2, and data exfiltration (visible handshake + flags).
- **UDP** is abused for amplification attacks, DNS tunneling, and fast exfil (harder to track because no handshake).

Key takeaway: Know which applications use **TCP** vs **UDP** by default. Anomalies (e.g., DNS over TCP) are often suspicious.

2. TCP Flags and Handshake

TCP Flags (important for detection):

- **SYN**: Start connection.
- **SYN-ACK**: Server response.
- **ACK**: Acknowledgment.
- **RST**: Reset/abort connection.
- **FIN**: Graceful close.

Normal Three-Way Handshake:

1. Client sends **SYN**.
2. Server replies **SYN-ACK**.
3. Client sends **ACK**.

Inspecting Flags in tcpdump:

Bash

```
tcpdump -nn -r file.pcap | more
```

Look for flags in square brackets: [S], [S.], [.].

Example command to see only SYN-ACK packets:

Bash

```
tcpdump -nn -r file.pcap 'tcp[tcpflags] == (tcp-syn|tcp-ack)'
```

Security relevance:

- **SYN scan** (half-open scan): Attacker sends many **SYN** packets but never completes handshake (no final ACK).
- Detection: High volume of **SYN** from one source with few/no **SYN-ACK** replies.

Demo insight:

- Normal web traffic: Clear handshake + TLS negotiation.
- Malicious: Many **SYN** packets to multiple IPs/ports without completing handshake → **SYN scanning**.

3. Ports and Ephemeral Ports

Ports:

- 16-bit numbers (0–65535).
- Well-known ports (0–1023): HTTP=80, HTTPS=443, DNS=53.
- Any port can host any service — attackers often use common ports (e.g., 443) to blend in.

Ephemeral ports:

- High-range ports (usually 49152–65535) used by clients for temporary connections.
- Client uses a new ephemeral port for each conversation.

Detection:

- Normal: Client uses many different ephemeral ports with low packet count per port.
- Suspicious: One ephemeral port with massive packet count → possible C2 beacon or long-lived connection.

Example commands (from demos):

Bash

```
tcpdump -nn -r file.pcap 'src host 192.168.218.131' | cut -f 3 -d " " | sort | uniq -c | sort -n
```

→ Shows ephemeral ports used by a client.

Bash

```
tcpdump -nn -r file.pcap 'src host 192.168.218.131' | cut -f 5 -d " " | cut -f 5 -d "." | sort | uniq -c | sort -n
```

→ Shows destination ports (services contacted).

Key takeaway: Look for unusual port usage or one ephemeral port dominating traffic.

4. Malicious Traffic Patterns (TCP/UDP)

TCP Anomalies:

- **SYN scan**: Many **SYN** packets, few/no **SYN-ACK** replies.
  - Command to detect:

    Bash

    ```
    tcpdump -nn -r syn-scan.pcap 'tcp[tcpflags] == tcp-syn' | cut -f 3 -d " " | cut -f 1-4 -d "." | sort | uniq -c | sort -n
    ```
  - Look for one source sending thousands of **SYN** to many IPs/ports.

UDP Anomalies:

- DNS over TCP (unusual — normally UDP).
- High-volume UDP to amplification targets (e.g., NTP, DNS).
- Unusual protocols on unexpected ports.

General Detection Tips:

- Baseline normal traffic (ports, flags, volume per port).
- Look for scripted behavior (same pattern repeated across many targets).
- Correlate with host logs (e.g., process that generated the traffic).

Demo insight (Malicious Traffic):

- SYN scanning: Single IP sending 143,738 SYN packets.
- Scanned ports: Many ports attempted 128 or 256 times.
- Internal IPs responding on unusual ports (135, 139, 445, 8080, 5357) → investigate as potential lateral movement.

Key takeaway: Understand normal protocol behavior (handshake, ports, flags). Anomalies (scans, unusual flags, high volume on one port) are strong indicators of malicious activity.

TCP DUMP Commands

Core Purpose: Capture, save, read, and filter live or saved network traffic for analysis.

Basic Commands (usable in any investigation):

```
sudo tcpdump                # Start live capture on default interface
sudo tcpdump -i eth0        # Capture on specific interface
sudo tcpdump -c 50          # Capture only 50 packets then stop
sudo tcpdump -w capture.pcap   # Write packets to file for later analysis
sudo tcpdump -r capture.pcap   # Read and display from saved pcap file
```

Common Filters (apply to live or saved captures):

```
sudo tcpdump host 192.168.1.100      # Traffic to/from a specific IP
sudo tcpdump port 80                 # Traffic on a specific port (HTTP example)
sudo tcpdump port 22                 # Traffic on SSH port
sudo tcpdump tcp                     # Only TCP packets
sudo tcpdump udp                     # Only UDP packets
sudo tcpdump icmp                    # Only ICMP (ping) packets
```

Verbose & Readable Output:

```
sudo tcpdump -vv                # Very verbose output (more packet details)
sudo tcpdump -A                 # Show packet data in ASCII (human readable)
sudo tcpdump -X                 # Show packet data in hex + ASCII
sudo tcpdump -nn                # Show IPs and ports as numbers only (no DNS resolution)
```

tcpdump + grep for Targeted Network Analysis (very useful for quick searches):

```
sudo tcpdump -r capture.pcap -nn -l | grep "HTTP"       # Find HTTP requests/responses
sudo tcpdump -r capture.pcap -nn -l | grep "POST"       # Find POST requests (possible exfil or C2)
sudo tcpdump -r capture.pcap -nn -l | grep "password"   # Search for plaintext passwords
sudo tcpdump -r capture.pcap -nn -l | grep "User-Agent"  # Spot unusual or suspicious User-Agents
sudo tcpdump -r capture.pcap -nn -l | grep -E "ru|cn|tk"  # Find suspicious country-code domains
sudo tcpdump -r capture.pcap -nn -l | grep "SYN"        # Find SYN packets (scans or floods)
```

Real-world Usage Tips:

- Always use -nn when piping to grep to avoid slow DNS lookups.
- Use -l (line-buffered) so grep processes output in real time.
- Combine with filters: sudo tcpdump -r capture.pcap host 192.168.1.100 and port 80 -nn -l | grep "POST"

Arkime (formerly Moloch) – Notes

4/20/26 6:05 PM

Arkime (formerly Moloch) – Notes
Purpose: Fast search and analysis of full network traffic (PCAPs) for hunting C2, phishing, exfil, and malware.
Main Interface:

- **Sessions tab**: Main view of all traffic sessions.
- **SPI View**: Expand a session for full details (headers, body, MD5).
- **Connections tab**: Visual graph of IP relationships.
- **Hunt tab**: Saved keyword searches across historical data.

Basic Workflow:

1. Ingest PCAP (script clears old data, captures, restarts viewer).
2. Adjust time range (top left: All or custom dates).
3. Click any value (port, IP, URI) to add as filter.
4. Use negation (and not) to remove known-good traffic.
5. Expand session (left icon) for deep details.

Analyzing Phishing Traffic

What to find: Malicious downloads via HTTP (few headers, disguised files).
Steps:

1. Click destination port **80** → add as filter (HTTP traffic).
2. In Information column, click URI → and not to remove known-good domains.
3. Expand suspicious session → check HTTP request (few headers = scripted).
4. Look at response: large databytes claiming PDF but showing PE (MZ header or "This program cannot be run in DOS mode").
5. Click **Show Images & Files** → download payload.
6. Use body MD5 for VirusTotal search.

Why: Minimal headers + disguised executables on port 80 is a common initial access method.

Detecting Malware Use of TLS Connections

What to find: Malware hiding C2 or exfil in TLS (often non-standard ports).
Steps:

1. Filter protocol to **TLS** + source IP.
2. Expand session → check certificate (self-signed or suspicious subject).
3. Note **JA3** hash → click to filter all sessions using same JA3.
4. Use Connections graph to see related IPs.

Why: JA3 fingerprints the TLS handshake even when data is encrypted, linking suspicious sessions.

Developing Techniques for Detecting Data Exfiltration

What to find: Stolen data leaving via SMTP, HTTP POST, or zipped files.
Steps:

1. Go to **Hunt** tab → Create search.
2. Search for keywords: "system info", "password", "cookie", "bank".
3. Set to source packets only (outbound exfil).
4. Expand matching session → **Show Images & Files** to download attachment.
5. For Base64: save content → decode with cat file.b64 | base64 -d > output.jpg.

Why: Hunt searches historical data for patterns; decoding reveals exactly what was stolen.

Quick Arkime Tips

- Click any field to filter (fastest way).
- Use **Connections** graph to see relationships.
- Check **body MD5** in SPI view for instant VT lookup.
- Export files directly from **Show Images & Files**.

Behavior Analysis

4/18/26 1:37 PM

Anomalies with Behavioral Analysis

Frequency → timing/volume
Protocol → rule deviations
Population → group outliers

1. Network Behavioral Analysis (NBAD) Fundamentals

What is NBAD? Behavioral analysis of network traffic to detect anomalies that signatures miss. Focuses on timing, volume, and patterns instead of static strings.
Why it matters: Signatures are fast but limited. Behavioral methods catch:

- Scans
- Beaconing/C2
- Brute force
- Tunneling
- Encrypted exfiltration

Key Data Sources:

- SPAN/mirror ports from switches (web servers, clients, domain).
- Tools: **Moloch** (PCAP storage + visualization), **Zeek/Bro** (protocol logs), **SiLK** (flow analysis), **Elasticsearch + Kibana** (dashboards).

Practice How You Play: Always baseline normal traffic before hunting anomalies.

2. Frequency Analysis

Core Idea: Compare how often or when events happen relative to normal behavior.
Useful Frequency Techniques:

- **Count per time unit**: Logins per minute, connections per second.
- **Percentage of total**: % of UDP vs TCP, % of traffic to one port.
- **Time between events**: Regular intervals (beaconing) vs human randomness.

What a Scan Looks Like:

- One-to-many: Single IP sending packets to many IPs/ports.
- Indicators: Short sessions (<1 second), many SYN without ACK, high volume from one source.
- Internal scan: Same pattern inside your network (higher risk).

Detection Example (Bro script for attempted connections):

- Threshold: 5+ attempted connections per minute from one origin.
- Use rwstats or Bro to count per source IP.

What Beaconing Looks Like:

- Regular, periodic connections to the same external IP/port.
- Indicators: Consistent timing, same packet sizes, low data volume.

Identifying Beaconing:

- Compare to normal web traffic: Normal has irregular timing; beaconing is perfectly spaced.
- Use Timelion/Kibana to visualize session duration over time.
- Filter short sessions or consistent intervals.

What Command & Control Looks Like:

- Malware checks in, receives commands, sends results.
- Often over HTTP/HTTPS or DNS to blend in.
- Indicators: Small, regular POSTs or queries; encoded data in subdomains.

Identifying C2:

- DNS C2: High-entropy subdomains, TXT/MX records, long queries.
- HTTP C2: Consistent short sessions, unusual JA3 hashes.
- Use entropy scoring (Bro script) or JA3 to spot outliers.

What Brute Force Looks Like:

- Rapid, repeated login attempts (username/password guessing).
- Indicators: Many short sessions to same port (RDP=3389, SSH=22), consistent packet counts, failed auth patterns.

Identifying Brute Force:

- RDP: 6–7 packets per failed attempt, short duration (~30ms).
- Success: Longer sessions with more packets.
- Use packet count + duration filters in Bro/SiLK.

Fitting Frequency Analysis into Daily Operations:

- Automate with cron jobs or tool APIs.
- Use for real-time alerts or historical hunts.
- Combine with other methods (protocol + population) for stronger signals.

3. Protocol Analysis

Core Idea: Understand normal protocol behavior per RFC, then spot deviations.
Following the Rules:

- Protocols must mostly follow standards to interoperate.
- Deviations (wrong flags, unusual record types, unexpected ports) = red flag.

Discovering DNS Command & Control:

- Normal DNS: Short queries, A/AAAA records, low volume.
- Malicious DNS: High-entropy subdomains, TXT records with large data, DNS over TCP for small queries.
- Detection: Entropy scoring (Bro script), unusual record types, direct queries to internal DNS from clients.

Deducing Activity of Encrypted Web Traffic (HTTPS):

- Normal: Varied timing, standard JA3 hashes, proper certificates.
- Malicious: Consistent short sessions, rare JA3 hashes, self-signed certs, unusual ports (e.g., 8888).
- Detection: JA3/JA3s hashes, certificate issuer checks, session duration analysis.

Analyzing SSH Authentication Behavior:

- Normal: Varied session lengths, standard key exchange.
- Brute force: Consistent packet count (~78 packets per failed attempt), short duration.
- Detection: Packet count per session + Bro SSH logs (auth success/failure).

Customizing DHCP Detection:

- Normal DORA: Discover → Offer → Request → ACK.
- Malicious: Rogue DHCP (bad options, exhaustion), unusual router/DNS settings.
- Detection: Hostname regex, router option validation, MAC randomization patterns.

Exploring Other Protocols:

- Focus on high-volume ones in your environment (LDAP, FTP, NTP, etc.).
- Apply same methods: baseline normal behavior, look for deviations in timing/volume/fields.

Key takeaway: Protocol analysis = know the rules, then find what breaks them.

4. Population Analysis

Core Idea: Compare behavior of one entity (IP, client) against the group (all clients).
Leveraging Machine Learning:

- Discover encrypted exfiltration by spotting outliers in HTTPS traffic volume or patterns.
- Tools: Elasticsearch machine learning jobs on client populations.

Investigating Client Device Relationship Anomalies:

- Look for unusual client-to-client communication (e.g., finance VLAN talking to engineering VLAN).
- Indicators: SMB transfers between unexpected subnets, RDP from scanning host.

Revealing Hidden Connections:

- Use connection graphs (Moloch) to visualize relationships.
- Combine with JA3, session duration, and frequency to link events.

Successfully Integrating Network Triage:

- Frequency → timing/volume anomalies.
- Protocol → rule deviations.
- Population → group outliers.
- Combine for stronger signals and fewer false positives.

Tuning Alerts:

- Whitelist known good JA3, hostname patterns, session behaviors.
- Aggregate alerts (e.g., one alert per 100 occurrences).
- Use dashboards (Kibana, Moloch) for visualization.

Recording Results:

- Use case management (TheHive) to link observables across alerts.
- Merge related cases when shared IPs, JA3, or files appear.

Putting It All in Perspective:

- NBAD fills gaps in signature-based detection.
- Still need host logs, application data, and machine data for full picture.
- Missing links: Process details, actual data moved, user actions.

Key takeaway: Behavioral analysis turns noise into signals. Combine frequency, protocol, and population methods with other data pillars for complete triage.

ZEEK Examples

4/21/26 7:56 PM

🧪 Example 1 — Detect Specific Bad Domains (Alert + Print)

```
@load base/frameworks/notice   # Load the Notice framework so we can generate alerts

# Create a global set (fast lookup list) of known bad domains
global bad_domains = {"bad.com", "evil.com"};

# This event runs every time a DNS request is seen
event dns_request(c: connection, msg: dns_msg, query: string)
{
   # Check if the queried domain exists in our bad_domains set
   if (query in bad_domains)
   {
      # Print message to console/log (for quick visibility/debugging)
      print fmt("Suspicious DNS query: %s", query);

      # Generate an alert using the Notice framework
      NOTICE([
         $note=Notice::PolicyViolation,  # Type/category of alert
         $msg=fmt("Detected bad domain: %s", query),  # Alert message with domain inserted
         $conn=c  # Attach connection info (IP, ports, etc.)
      ]);
   }
}
```

🧪 Example 2 — Detect Possible DNS Tunneling (Long Queries)

```
@load base/frameworks/notice   # Load alerting framework

# Triggered on every DNS request
event dns_request(c: connection, msg: dns_msg, query: string)
{
   # Check if the domain length is greater than 50 characters
   if (|query| > 50)   # |query| means "length of the string"
   {
      # Print suspicious long query
      print fmt("Long DNS query detected: %s", query);

      # Generate alert for possible DNS tunneling
      NOTICE([
         $note=Notice::PolicyViolation,  # Alert type
         $msg=fmt("Possible DNS tunneling: %s", query),  # Alert message
         $conn=c  # Include connection details
      ]);
   }
}
```

🧪 Example 3 — Track Unique Domains per Host (High Activity Detection)

```
# Create a table:
# Key = IP address (addr)
# Value = set of domains (unique values only)
global dns_tracker: table[addr] of set[string];

# Runs on every DNS request
event dns_request(c: connection, msg: dns_msg, query: string)
{
   # Add the queried domain to this host's set
   dns_tracker[c$id$orig_h] += query;
   # c$id$orig_h = source IP address

   # Check how many unique domains this IP has queried
   if (|dns_tracker[c$id$orig_h]| > 100)  # |set| = number of elements
   {
      # Print alert for high DNS activity
      print fmt("High DNS activity from: %s", c$id$orig_h);
   }
}
```

🧪 Example 4 — Custom DNS Logging (Create Your Own Log File)

```
module DNSLog;  # Define a module (namespace) to organize this script

export {
   # Register a new log type called LOG
   redef enum Log::ID += { LOG };

   # Define the structure (columns) of our log
   type Info: record {
      ts: time;       # Timestamp of event
      src_ip: addr;   # Source IP address
      domain: string;  # Queried domain
   };
}

# Runs when Zeek starts
event zeek_init()
{
   # Create the log stream using the structure defined above
   Log::create_stream(LOG, [$columns=Info]);
}

# Runs on every DNS request
event dns_request(c: connection, msg: dns_msg, query: string)
{
   # Write a log entry
   Log::write(LOG, [
      $ts=network_time(),       # Current network timestamp
      $src_ip=c$id$orig_h,      # Source IP
      $domain=query             # Domain queried
   ]);
}
```

🧪 Example 5 — Detect SSH Brute Force Attempts (Your Custom One)

```
@load base/frameworks/notice       # Load alerting framework
@load base/protocols/ssh           # Load SSH protocol analyzer

# Define a new Notice type specifically for SSH brute force
redef enum Notice::Type += { SSH_Brute_Force };

# Create a global table:
# Key = IP address
# Value = count of failed SSH attempts
global ssh_attempts: table[addr] of count &default=0;
# &default=0 means every new IP starts with 0 attempts automatically

# This event runs every time an SSH authentication attempt happens
event ssh_auth_attempt(c: connection, success: bool)
{
   # Check if login attempt FAILED
   if (!success)   # ! means "NOT" → so this means "if login failed"
   {
      # Increase the failed attempt count for this IP
      ssh_attempts[c$id$orig_h] += 1;

      # If failed attempts exceed 5
      if (ssh_attempts[c$id$orig_h] > 5)
      {
         # Generate alert for brute force detection
         NOTICE([
            $note=SSH_Brute_Force,  # Custom alert type we defined above

            # Message explaining what happened
            # %s = placeholder for string (IP gets inserted)
            $msg=fmt("Excessive failed SSH attempts from %s",
                  c$id$orig_h),

            # Attach connection details (VERY IMPORTANT in real-world)
            $conn=c
         ]);
      }
   }
}
```

Wireshark

4/18/26 6:32 PM

Identify Common Cyber Network Attacks with Wireshark

Core Concept: Wireshark is a **packet-level microscope**. Use it **after** you have a lead from IDS alerts, firewall logs, or server event logs. Do not start by randomly capturing everything — that wastes time and misses key traffic.
When to use Wireshark:
- After an IDS alert (Suricata/Snort/Zeek).
- After firewall or server log events show suspicious activity.
- To map activity to **MITRE ATT&CK** (Reconnaissance → Initial Access → Execution → etc.) or **Cyber Kill Chain**.

Best Practice:
- Capture 24/7 at key points (internet ingress, core server links).
- Always start from a specific lead (IP, port, time, event) instead of full capture.

Module 3: Analyzing Port Scans and Enumeration Methods

Core Concepts: Attackers first discover hosts and open ports before exploiting them. Common methods: ARP scans, ICMP ping sweeps, TCP SYN scans, OS fingerprinting, HTTP path enumeration.

Lab 1 – Detecting Network Discovery Scans

- Filter: 'arp'.
- Look for one source MAC sending ARP requests for many IPs in the subnet (one-to-many pattern).
- Right-click ARP request → Prepare as Filter → Selected → change opcode to '2' for replies.
- ARP replies reveal exactly which hosts are active on the network — the first step in attacker reconnaissance.

Lab 2 – Identifying Port Scans (Part 1 & 2)

- Filter: 'tcp.flags.syn == 1 && tcp.flags.ack == 0' (SYN-only packets).
- Go to Statistics → Conversations → TCP tab → sort by Address B or Port B.
- Look for one source IP sending SYNs to many different ports/IPs.
- To see open ports: 'tcp.flags.syn == 1 && tcp.flags.ack == 1'.
- Save filter under TCP → Open Ports.
- SYN scans are the most common because they are fast and do not complete the handshake, making them harder for basic firewalls to log.

Lab 3 – Analyzing Malware for Network and Port Scans (Part 1 & 2)

- Filter: 'arp' → find source doing ARP sweep across subnet.
- Filter: 'icmp' → see ping sweep.
- Filter: 'tcp.flags.syn == 1 && tcp.flags.ack == 0' → see TCP SYN activity after discovery.
- Use Statistics → Conversations → IPv4 and TCP to see one-to-many patterns.
- Malware often performs the exact same discovery scans as manual tools like Nmap, making the behavior easy to spot once you know what to filter.

Lab 4 – Detecting OS Fingerprinting (Part 1 & 2)

- Filter: '!arp' to remove noise.
- Look for many crafted TCP SYNs from one source with unusual TCP options or TTL values.
- Add column: Right-click IP → Time to Live → Apply as Column.
- Look for TTL values jumping around (30–50 range is suspicious locally).
- Filter for Xmas scan: 'tcp.flags == 0x029' (URG+PSH+FIN set — "lights up like a Christmas tree").
- Filter for Null scan: 'tcp.flags == 0x000' (no flags set).
- Xmas and Null scans send malformed packets to see how the target OS stack responds, helping identify the OS version.

Lab 5 – Analyzing HTTP Path Enumeration

- Filter: 'http.request.method == "GET"'.
- Look for many sequential or dictionary-like GETs to different paths.
- Look for high number of 404 Not Found responses.
- Check User-Agent column for 'gobuster' (common enumeration tool).
- Gobuster is a popular open-source tool attackers use to brute-force hidden directories and files on web servers.

Module 4: Analyzing Common Attack Signatures of Suspect Traffic

Lab 6 – Analyzing TCP SYN Attacks

- Filter: 'tcp.flags.syn == 1 && tcp.flags.ack == 0'.
- Go to Statistics → Conversations → TCP → sort by Address B or Port B.
- Look for many SYNs from many sources to one target.
- Filter for responses: 'tcp.flags.syn == 1 && tcp.flags.ack == 1 && ip.dst == <target>'.
- SYN floods overwhelm the target's resources because it must allocate memory for every half-open connection.

Lab 7 – Spotting Suspect Country Codes with GeoIP

- Filter: 'ip.geoip.country == "Russia"' (or China, etc.).
- Go to Statistics → Endpoints → look at GeoIP columns.
- Filter for outbound: 'ip.geoip.country == "Russia" && ip.src == <internal range>'.

Lab 8 – Filtering for Unusual Domain Name Lookups

- Filter: 'dns.qry.name contains "hackersden"'.
- Use regex for multiple countries: 'dns.qry.name matches "(ru|cn|jp|uk)"'.
- Unusual country-code domains are a quick indicator of reconnaissance or C2 domains attackers commonly register.

Lab 9 – Analyzing HTTP Traffic and Unencrypted File Transfers

- Filter: 'http.request.method == "GET" || http.request.method == "POST"'.
- Look for .exe, .bin, .php in Request URI.
- Go to File → Export Objects → HTTP to extract suspicious files.
- Filter for executables: 'http.request.uri matches "(\.exe|\.bin|\.php|\.zip)"'.
- Attackers disguise executables as .png or other harmless extensions to bypass basic filters.

Lab 10 – Analysis of a Brute Force Attack

- Filter: 'ftp' or 'tcp.port == 21'.
- Follow TCP Stream on login attempts.
- Look for many "530 Login incorrect" responses followed by one "230 Login successful".
- Response code 230 means successful FTP login — the moment the attacker gains access.

Module 5: Identifying Common Malware Behavior

Lab 11 – Malware Analysis with Wireshark (Part 1 & 2)

- Filter: 'http.request || http.response'.
- Look for POSTs to unusual countries.
- Follow TCP Stream on POSTs → look for stolen data (usernames, passwords, PC name).
- Go to File → Export Objects → HTTP → look for .exe disguised as .png.
- Malware often hides executables inside image-like filenames (e.g., cursor.png) to blend with normal web traffic.

Module 6: Identify Shell, Reverse Shell, Botnet, and DDoS Attack Traffic

Lab 12 – Analyzing Reverse Shell Behavior

- Filter: 'tcp.port == 1337' (common reverse shell port).
- Follow TCP Stream → look for command-like traffic (ls, whoami, cd).
- Reverse shell works by planting a PHP script on a vulnerable web server that forces the server to connect back to the attacker on the chosen port.

Lab 13 – Identifying Botnet Traffic (Part 1 & 2)

- Filter: http.request || http.response.
- Look for many GETs/POSTs from one client to unusual countries.
- Follow TCP Stream on POSTs → look for process lists, PC name, credentials.
- Use tcp.analysis.flags to spot retransmissions and unusual TCP behavior that stands out from normal traffic.
- Botnet C2 traffic often uses HTTP POSTs to phone home with system information while blending with normal web traffic.

Lab 14 – Analyzing Data Exfiltration with Wireshark

- Filter: ftp or tcp.port == 21.
- Follow TCP Stream on login → look for "STOR" (upload) commands.
- Look for large outbound transfers after malware download.
- After downloading the .exe, the client waits ~30 minutes then uploads stolen data (usernames, passwords, banking info) via FTP STOR commands in cleartext.

Investigate Cyber Network Attacks with Wireshark LAB:

Analyzing Command & Control (C2) Infrastructure

Concept: C2 traffic is the ongoing communication between a compromised host and the attacker's server for commands, status checks, and data exfil. It often tries to blend in with normal traffic.
What to look for:
- Beaconing pattern: packets sent at very regular intervals (e.g., every 5.03 seconds).
- Consistent small packet sizes across many sessions.
- Communication on port 80 (HTTP) with encrypted payload (random bytes in Packet Bytes pane, not readable HTTP).
- No normal HTTP requests/responses even though port 80 is used.

Steps in Wireshark:
1. Open Conversations (Statistics → Conversations → TCP tab).
2. Check **Rel. Start** column for regular timing between sessions.
3. Check packet **Length** column for consistent small sizes.
4. Select a session → look in **Packet Bytes** pane for random/encrypted data (no readable HTTP headers).
5. Compare with normal HTTP traffic (tcp.stream eq X) to see missing HTTP layer.

Why it matters: Regular timing + small consistent packets + encrypted data on port 80 is a strong indicator of beaconing C2, even when it mimics web traffic.

Investigating SSH Brute Force Attack

Concept: Brute force attempts many username/password combinations rapidly against SSH (port 22).
What to look for:
- Many short TCP sessions from the same source to port 22.
- Rapid repeated connection attempts (visible in Flow Graph or Conversations).
- Consistent short duration and packet counts per failed attempt.
- No successful key exchange or very quick resets.

Steps in Wireshark:
1. Filter: tcp.port == 22.
2. Open **Statistics → Conversations → TCP** → sort by Address or Port.
3. Open **Statistics → Flow Graph** to visualize repeated short connection attempts.
4. Look at **Time** column for attempts happening in ~0.3 seconds intervals.

Why it matters: Legitimate users rarely attempt logins this quickly and repeatedly. The pattern stands out clearly in Conversations and Flow Graph.

Investigating C2 from a Metasploit Server (Meterpreter)

Concept: Meterpreter C2 often uses encrypted channels that look like normal traffic but show unusual session behavior.
What to look for:
- Very small packet lengths and short session durations.
- Communication on port 80 with no actual HTTP protocol data (no GET/POST headers).
- Low percentage of HTTP in Protocol Hierarchy despite using port 80.
- Consistent small back-and-forth packets (beaconing).

Steps in Wireshark:
1. Open **Statistics → Conversations → TCP** → look for sessions with tiny byte counts.
2. Filter: tcp.stream eq 3 (or the suspicious stream number).
3. Check **Protocol Hierarchy** (Statistics → Protocol Hierarchy) → HTTP should be near 0% for that stream.
4. Look in Packet Details for missing HTTP layer even on port 80 traffic.

Why it matters: Attackers use port 80 to blend in, but the lack of real HTTP and the regular small-packet pattern reveals C2 activity.

DNS

4/15/26 9:42 PM

Summary:

- **Normal DNS**: Low volume, simple domains, UDP port 53, expected record types.
- **Malicious DNS**: High-entropy subdomains, periodic beaconing, tunneling via long queries, or DNS over TCP for small data.
- **Detection workflow**:
  1. Capture traffic (**Wireshark** / **tcpdump** / **Zeek**).
  2. Filter DNS (dns in Wireshark).
  3. Check entropy, volume, and patterns.
  4. Correlate with host logs (processes making DNS requests).
  5. Investigate root cause and block suspicious domains.

Common Tools & Commands:

- **Wireshark**: Filter dns, inspect queries/responses.
- **tcpdump**: tcpdump -nn port 53.
- **Zeek**: Process PCAP for dns.log and entropy analysis.
- **Hosts file check**: Look for unexpected entries.

1. DNS Basics

What DNS does: Translates human-readable domain names (e.g., **www.example.com**) into IP addresses (e.g., **93.184.216.34**).

Query/Response Flow:

- Client sends **DNS query** (usually over **UDP** port 53).
- Resolver (local or recursive server) looks up the record.
- Server returns **DNS response** with the IP (A record) or other data.

Common Record Types:

- **A** — IPv4 address.
- **AAAA** — IPv6 address.
- **CNAME** — alias to another name.
- **MX** — mail server.
- **TXT** — text (often used for verification or malicious data).

Security relevance: DNS is trusted and rarely blocked. Attackers abuse it for:

- **C2 communication** (malware queries attacker-controlled domains).
- **DNS tunneling** (hide data inside DNS queries/responses).
- **Exfiltration** (encode stolen data in subdomains).

Normal vs. Malicious:

- Normal: Short, simple queries; low volume; standard record types.
- Malicious: High volume, high-entropy subdomains, unusual record types, or DNS over TCP (rare for normal use).

Key takeaway: DNS is a common covert channel. Monitor for anomalies in query volume, subdomain patterns, and entropy.

2. Creating and Inspecting Normal DNS Traffic

Generating traffic (demo approach):

- Use browser or tools like **curl** to trigger DNS queries.
- Capture with **Wireshark** or **tcpdump** on the interface.

Example tcpdump command:

Bash

```
tcpdump -nn -i eth0 port 53
```

Inspecting in Wireshark:

- Filter: dns
- Look at:
  - **Query** section: Domain name, record type (A, AAAA, etc.).
  - **Response** section: Resolved IP, TTL.

UDP vs TCP for DNS:

- **UDP** (default, port 53): Fast, connectionless — used for most queries.
- **TCP** (port 53): Used for large responses (e.g., zone transfers) or when UDP is blocked.
- Malicious indicator: DNS over TCP for small queries → possible tunneling.

Key takeaway: Baseline normal DNS (low volume, simple domains). Any deviation (high volume, encoded subdomains) is suspicious.

3. Modifying Traffic Flow & Hosts File

Hosts File:

- Local file that overrides DNS (maps domain to IP manually).
- Location: C:\Windows\System32\drivers\etc\hosts (Windows) or /etc/hosts (Linux).

Security relevance:

- Attackers modify hosts file for redirection (phishing, C2 hiding).
- Detection: Unexpected entries pointing to suspicious IPs.

Demo insight:

- Modifying hosts file changes resolution without querying real DNS.
- Useful for testing or attacker persistence (bypass DNS monitoring).

Key takeaway: Check hosts file during host forensics — it can bypass network DNS controls.

4. Detecting Malicious DNS (C2 and Tunneling)

DNS C2:

- Malware uses DNS queries to communicate with attacker (beaconing, commands in subdomains).
- Indicators: Periodic queries to random-looking domains, high entropy in subdomains.

DNS Tunneling:

- Hide data inside DNS queries/responses (encode payloads in subdomains).
- Indicators: Very long subdomains, high entropy, unusual volume, or TXT records with large data.

Detection Techniques:

- **Entropy score**: High randomness in subdomain names (use **Zeek** scripts).
- Volume: Sudden spike in DNS queries from one host.
- Pattern: Queries without corresponding normal web traffic.

Demo insight (Generating Entropy Score with Zeek):

- **Zeek** processes PCAP and generates logs (dns.log).
- Look for high-entropy subdomains or unusual query types.
- Command example (simplified):
  Bash
  ```
  zeek -r capture.pcap
  ```
- Review dns.log for suspicious entries.

Advanced Analysis:

- Correlate DNS with host processes (e.g., malware making repeated queries).
- Baseline normal DNS behavior in your environment.
- Use Suricata rules or SIEM to alert on high-entropy or anomalous DNS.

Key takeaway: DNS is stealthy. Look for **high entropy**, unusual volume, or DNS without normal application traffic as strong C2/tunneling indicators.